

Abstract

Contents

Abstract	1
1 Introduction	5
1.1 Context and Motivation	5
1.2 Research Objectives	5
1.3 Main Contributions of the Thesis	5
1.4 Structure of the Document	5
2 Theoretical Background	6
2.1 The Evolution of Learning in Artificial Intelligence	6
2.1.1 Artificial Intelligence Paradigms	6
2.1.2 Symbolic and Hybrid Approaches in Artificial Intelligence	8
2.1.3 Reinforcement Learning: Learning Through Experience	9
2.2 Computer Vision Techniques	10
2.2.1 Classical Techniques for Video Analysis	10
2.2.2 Visual Perception in Dynamic Environments	10
2.2.3 Deep Learning for Feature Extraction from Video Streams	10
2.3 Knowledge Representation and Reasoning	10
2.3.1 Symbolic Representations, Semantic Networks, and Interpretable Models	10
2.4 Reinforcement Learning Principles	10
2.4.1 Theoretical foundations	10
2.4.2 Types of Reinforcement Learning	13
2.4.3 Deep Q-Networks	16
2.5 Atari-like Environments as Testbeds	16
2.5.1 Characteristics and Challenges of 2D Game Environments	16

3	State of the Art and Challenges	17
3.1	Challenges in Symbolic Interpretation	18
3.1.1	The Illusion of Understanding	18
3.1.2	The Black Box Problem	18
3.1.3	Learning Paradigms and Explainable AI	18
3.2	Cognitive Foundations	18
3.2.1	Human World Modelling and the Origins of Structured Cognition	18
3.2.2	Core Knowledge Theory: Cognitive Principles and Psychological Foundations	18
3.2.3	Definition and Relevance for World Understanding	18
3.2.4	Cognitive Models and Symbolic Representation	18
3.3	Positioning of the Proposed Framework	18
4	Framework Design - Proposed Framework	19
4.1	Conceptual Foundations	19
4.1.1	Evolution of the Idea	19
4.2	Proposed Implementation	19
4.2.1	Segmentation	19
4.2.2	Symbolic System as Knowledge Construction Module	19
4.2.3	From Symbolic Rules to a Playable Environment	19
4.2.4	Generic Environment as a Transitional Representation	19
4.2.5	Deep Q-Network in a Domain-Agnostic Environment	19
5	Implementation	20
5.1	End-to-End Pipeline	20
5.2	General Architecture of the Framework	20
5.2.1	Design Choices	20
5.2.2	Selected Techniques and Algorithms	20
5.2.3	Modular Structure of the System	20
5.2.4	Optimizations and Efficiency Considerations	20
6	Experimental Evaluation	21
6.1	Datasets and Testing Scenarios	21
6.1.1	Evaluation Metrics: Accuracy, Interpretability, Scalability	21
6.2	Experimental Results	21
6.2.1	Comparison with Existing Methods	21
6.2.2	Critical Discussion of the Results	21

7 Applications and Future Perspectives – Conclusion	22
7.1 Future Directions	22
7.1.1 Potential Applications of the Framework	22
7.1.2 Possible Extensions of the Work	22
7.1.3 Open Challenges and Future Research Directions	22
7.2 Summary of Contributions, Impact of the Research, and Final Reflections	22
References	23
A Additional Technical Details	26
B Code Snippets or Pseudocode of Relevant Modules	27
C Dataset Specifications or Experimental Settings	28

1 Introduction

1.1 Context and Motivation

1.2 Research Objectives

1.3 Main Contributions of the Thesis

1.4 Structure of the Document

2 Theoretical Background

The field of artificial intelligence has undergone deep transformations since its inception, evolving through distinct paradigms that reflect different philosophical and technical approaches to machine intelligence. This chapter provides the theoretical foundations necessary to understand the framework proposed in this thesis, exploring the evolution of learning paradigms in AI, some techniques of visual perception and the principles underlying symbolic reasoning and reinforcement learning.

2.1 The Evolution of Learning in Artificial Intelligence

2.1.1 Artificial Intelligence Paradigms

The history of artificial intelligence is characterized by the emergence and evolution of three major paradigms, each representing a distinct approach to providing machines the human-like reasoning abilities. The symbolic paradigm, dominant from the 1950s through the 1980s. This paradigm was grounded in the hypothesis that human intelligence could be replicated through the manipulation of symbols, according to logical rules, to perform reasoning tasks. The systems built on this paradigm operated on explicit knowledge representations, using formal logic and rule-based reasoning to solve problems. These systems are grounded in formal logic and manually designed rule sets. Classic examples include expert systems such as MYCIN and Deep Blue, as well as semantic networks and logic programming frameworks [1]. They are capable of performing reasoning tasks and have achieved notable success in constrained domains where knowledge can be explicitly encoded. However, the symbolic paradigm faced significant limitations due to its high reliance on manually encoded knowledge. Indeed, when faced with tasks requiring pattern recognition, learning from experience, the management of uncertainty or incomplete information, it suffered from brittle-

ness [1, 2]. These challenges gave rise to the connectionist paradigm, which gained prominence in the late 1980s and experienced a resurgence in the 2010s with the advent of deep learning [3]. Inspired by the structure and function of biological neural networks, connectionist systems learn representations directly from data through training processes that adjust connection weights between artificial neurons [3, 4]. This paradigm has achieved remarkable success in tasks such as image recognition, natural language processing and game playing, often surpassing human-level performance in specific domains [5]. Their capacity for robust and interpretable reasoning remains limited. Deep learning still struggles with structured reasoning, causal inference and factual consistency. Despite the impressive capabilities of neural networks, their operation and internal representation often lacks transparency, giving rise to what is commonly referred to as the "black box problem" [2], which limits interpretability, accountability and trust. Indeed, many of these approaches compromised interpretability and relied on strong assumptions. Moreover, they lacked the rich semantic expressiveness characteristic of symbolic systems. These limitations provide strong motivation for integrating neural and symbolic paradigms. In short, symbolic systems excel at reasoning but lack at perception, while neural networks excel at perception but struggle with reasoning [1]. This limitation has motivated the emergence of a hybrid paradigm, which seeks to combine the learning capabilities of neural networks with the interpretability and reasoning power of symbolic systems. Neuro-symbolic AI represents this third wave, aiming to bridge the gap between data-driven learning and knowledge-based reasoning as well as between logic-based inference and gradient-based learning. By integrating these paradigms, it aims to potentially offer systems that are both powerful and explainable [6]. Neuro-symbolic methods have demonstrated significant promise across a wide range of application areas, especially in scenarios that demand the combination of data-driven learning and structured symbolic reasoning [6]. This emerging paradigm reflects a synthesis of complementary approaches, bringing together the statistical strengths of deep learning, the formal foundations of symbolic logic and the adaptability of large-scale pretrained models.

Nonetheless, several important challenges remain. These include:

- Ensuring compatibility between the discrete symbolic structures used in symbolic approaches and the continuous vector representations typical of neural networks.
- Enabling the model to dynamically learn, adapt, modify, and refine logical rules.

- Achieving an optimal balance between system complexity and computational efficiency.
- Addressing the absence of unified architectural frameworks by developing a general architecture that systematically and reusably integrates neural and symbolic components.

Although several challenges persist, neuro-symbolic AI is widely regarded as a promising direction for developing systems capable of transparent, reliable and generalizable reasoning.

2.1.2 Symbolic and Hybrid Approaches in Artificial Intelligence

Symbolic AI operates on the principle that intelligence can be reduced to the manipulation of symbols representing concepts, objects and relationships. The knowledge in symbolic systems is typically encoded using various representation schemes, including production rules, semantic networks, frames and logical predicates. These representations enable explicit reasoning processes such as deduction, induction and abduction, allowing systems to draw inferences, explain their decisions and incorporate domain knowledge provided by human experts. The advantages of symbolic approaches include interpretability, the ability to incorporate prior knowledge and support for logical reasoning and explanation. However, they also present significant challenges: knowledge acquisition bottleneck, brittleness when facing situations not covered by explicit rule and difficulty in learning from raw sensory data. The manual encoding of knowledge is intensive and may not capture the full complexity of real-world domains [1]. Hybrid approaches seek to overcome these limitations by integrating symbolic and connectionist methods. Various integration strategies have been proposed, including systems where neural networks generate symbolic representations that are then processed by reasoning engines, architectures that embed symbolic knowledge into neural network structures and frameworks that use symbolic reasoning to guide neural learning processes [6]. Recent work in neuro-symbolic AI has demonstrated promising results in domains requiring both pattern recognition and logical reasoning, such as visual question answering, knowledge graph completion, and program synthesis. Following the advent of neuro-symbolic artificial intelligence, a paradigm shift has been induced in the way machines learn. This approach combines deductive, rule-based learning, typical of symbolic artificial intelligence, with the inductive and pattern-recognition capabilities characteristic of neural networks. The result is a

hybrid approach that leverages the strengths of both domains, leading to a more comprehensive learning methodology. Neuro-symbolic concepts demonstrate strong compositional generalization: new concepts can be created by structurally combining previously learned or existing ones [7]. This compositionality naturally allows for a decomposition of the learning problem, in order to learn and recognize basic concepts. During the inference phase, these concepts can be recombined to achieve the objectives of more complex tasks. Neuro-symbolic AI is one of the most exciting directions in AI research today. It aims to create intelligent systems capable of learning and reasoning like humans.

2.1.3 Reinforcement Learning: Learning Through Experience

Reinforcement learning (RL) represents a distinct learning paradigm in which an agent learns to make decisions by interacting with an environment. Unlike supervised learning, which requires labeled training data, or unsupervised learning, which seeks to discover patterns in unlabeled data, reinforcement learning is based on the principle of learning from the consequences of actions through trial and error [8]. The agent takes actions, observes their consequences, and receives feedback in the form of rewards. At the beginning of learning, an agent may have limited or no prior knowledge of the environment. In model-free approaches, no explicit model is available, and learning relies entirely on interaction. In contrast, model-based methods assume a known model or aim to learn it explicitly. This flexibility makes reinforcement learning suitable for complex environments, where providing a complete dataset or manually specifying optimal behavior is impractical. The core structure of Reinforcement Learning (RL) is modelled through a Markov Decision Process (MDP), which provides a mathematical framework for sequential decision-making [8]. An MDP is defined by a finite or continuous set of states representing the possible situations an agent can encounter, a set of actions available to the agent and a transition function that specifies the probability of moving from one state to another after taking a particular action. Additionally, a reward function assigns numerical feedback to each state-action pair, quantifying the immediate benefit of a given choice. By observing the current state, selecting an action, and receiving a reward and a new state, the agent incrementally improves its decision-making strategy [9]. Over time, the agent aims to learn an optimal policy, which is a strategy mapping states to actions. The goal of this policy is to maximize the expected cumulative reward across future interactions, balancing short-term

gains with long-term benefits through exploration and exploitation. Several agent design approaches exist, including methods based on value estimation and approaches that learn action values directly, such as Q-learning. More advanced techniques combine different strategies to improve learning efficiency. Due to its ability to learn from experience, reinforcement learning has achieved notable success in domains such as game playing, robotics, and resource management [9].

2.2 Computer Vision Techniques

2.2.1 Classical Techniques for Video Analysis

2.2.2 Visual Perception in Dynamic Environments

2.2.3 Deep Learning for Feature Extraction from Video Streams

2.3 Knowledge Representation and Reasoning

2.3.1 Symbolic Representations, Semantic Networks, and Interpretable Models

2.4 Reinforcement Learning Principles

This section delves deeper into the principles and methods of reinforcement learning, with particular focus on Deep Q-Networks, which play a central role in the framework proposed in this thesis.

2.4.1 Theoretical foundations

The mathematical foundation of reinforcement learning rests on the framework of Markov Decision Processes (MDPs). An MDP formalizes sequential decision-making problems in which an agent interacts with an uncertain environment and seeks to maximize a cumulative reward over time. An MDP is defined by the tuple (S, A, P, R, γ) , where:

- S is the state space, i.e., the set of all possible configurations of the environment.

- A is the action space, i.e., the set of all actions available for the agent in each state.
- $P(s' \mid s, a)$ is the transition probability function, i.e., the probability that, after performing action a in state s , the system transitions to state s' .
- $R(s, a)$ is the reward function, which provides a numerical value quantifying the desirability of transitioning from state s to state s' after performing action a , assigning a scalar value to each state-action pair.
- $\gamma \in [0, 1]$ is the discount factor that determines the relative importance of immediate versus future rewards. Smaller values of γ encourage myopic behavior focused on immediate gains, whereas values closer to 1 promote long-term planning.

A fundamental assumption in MDPs is the Markov property, which states that the future evolution of the system depends solely on the current state and action, independent of the full history of past states. Mathematically:

$$P(s' \mid s, a, s_{t-1}, a_{t-1}, \dots) = P(s' \mid s, a)$$

This memoryless characteristic significantly simplifies the analysis and allows the use of dynamic programming techniques. The agent tries to maximize the total cumulative reward. This cumulative reward is defined as:

$$G_t = \sum_{k=0}^{\infty} \gamma^k R(s_{t+k}, a_{t+k})$$

It is not simply the sum of instant rewards, but the discounted reward over time. Due to the γ it is possible to give more weight to immediate or future rewards.

Policie

In reinforcement learning, a policy π defines the agent's behavior by specifying which action should be taken in each possible state [9]. Formally, a policy can be represented either as a function or as a probability distribution over actions. Two main types of policies can be distinguished. A deterministic policy $\pi(s)$ assigns a single, fixed action to each state. In contrast, a stochastic policy $\pi(a \mid s)$ associates a probability with each available action. Agents progressively learn an optimal policy through continuous interaction with the environment. This learning process requires collecting experience by visiting

new states and testing different actions. Through this interaction, the agent refines its strategy in order to maximize the cumulative reward obtained over time. A central challenge in this process is the exploration-exploitation trade-off. Exploration encourages the agent to try new and unfamiliar actions in order to discover potentially better rewards and improve its knowledge of the environment. On the other hand, exploitation focuses on selecting actions that are already known to yield high returns, thus capitalizing on past experience. Achieving an effective balance between these two objectives is crucial for learning a robust and optimal policy.

Value Function

Value functions are used to evaluate policies and play a fundamental role in many reinforcement learning algorithms. The state-value function $V_\pi(s)$ estimates the expected cumulative reward obtained when starting from state s and following policy π .

$$V_\pi(s) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t, s_{t+1}) \right]$$

The action-value function $Q_\pi(s, a)$ measures the expected return after taking action a in state s and subsequently following policy π , are also called Q-values [10].

$$Q_\pi(s, a) = \mathbb{E} [R(s, a, s') + \gamma V_\pi(s')]$$

The central objective of reinforcement learning is to identify an optimal policy π^* that maximizes these value functions across all states.

Potential and areas of application

MDPs provide the theoretical foundation for a wide range of reinforcement learning algorithms, including value iteration, policy iteration, temporal-difference learning and Q-learning. These methods exploit the mathematical structure of MDPs to iteratively improve policies, either when the environment model is known or when it must be learned from experience. As a result, MDPs play a crucial role in enabling autonomous agents to learn effective behaviors through interaction with complex and uncertain environments. Due to their strong theoretical foundation and flexibility, MDPs are extremely important across numerous application domains. In robotics and automation, for example, robots rely on MDPs to navigate uncertain and dynamic environments. Beyond robotics, MDPs are also widely applied in healthcare for treatment planning and optimization, in finance for trading

strategies and risk management, and in game AI for developing intelligent and adaptive agents. This broad applicability highlights the practical significance of MDPs in real-world decision-making problems [9].

The path to success

The primary objective of reinforcement learning is to identify an optimal policy that maximizes long-term rewards. This objective leads to one of the most fundamental results in RL theory: the Bellman Optimality Equation [11]. This equation provides a recursive definition of optimal value functions, both for states and for state-action pairs [12].

- **Optimal State Value Function** $V^*(s)$:

$$V^*(s) = \max_a \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma V^*(s')] \quad (2.1)$$

- **Optimal Action Value Function** $Q^*(s, a)$:

$$Q^*(s, a) = \sum_{s'} P(s' | s, a) \left[R(s, a, s') + \gamma \max_{a'} Q^*(s', a') \right] \quad (2.2)$$

The Bellman optimality equation removes any dependence on a particular strategy. Instead, it establishes a self-consistency condition that optimal value functions must satisfy. Specifically, the optimal state-value function is defined as the maximum expected return achievable by selecting the best possible action in a given state. Similarly, the optimal action-value function is expressed as the immediate reward plus the highest expected future return obtainable from the subsequent state [12].

The Bellman Optimality Equation serves as the theoretical cornerstone for a wide range of optimal control algorithms used in reinforcement learning.

2.4.2 Types of Reinforcement Learning

Several methods have been developed to solve Markov Decision Processes (MDPs), differing mainly in how they use information about the environment. These approaches can be broadly classified into model-based and model-free methods. Model-based techniques rely on a known or learned model of the environment to predict future states and rewards, allowing agents to plan their actions in advance. In contrast, model-free methods do not assume any prior knowledge of the environment and instead learn optimal behaviors directly through experience and interaction. Understanding this distinction is fundamental for analyzing how different reinforcement learning algorithms operate and adapt to uncertainty.

Model-Based Reinforcement Learning

Dynamic Programming Methods Dynamic programming is a model-based approach and solves MDPs by iteratively improving value functions through:

- **Policy Iteration:** alternates between policy evaluation and policy improvement until the policy converges to the optimal policy:
 - *Policy evaluation* consists of estimating the value function associated with a given policy π , typically by computing the expected return obtained when the agent follows that policy from each state. This step answers the question of how good the current policy is, by assessing its long-term performance in the environment.
 - *Policy improvement*, on the other hand, aims to enhance the current policy by exploiting the information obtained during policy evaluation. In this phase, the policy is updated by selecting actions that yield higher expected returns according to the evaluated value function, resulting in a new policy that is guaranteed to be at least as good as the previous one [12].
- **Value Iteration:** updates the value function until it converges to the optimal value function.

$$V_{k+1}(s) = \max_{a \in A} \sum_{s' \in S} P(s' | s, a) [R(s, a, s') + \gamma V_k(s')]$$

Dynamic programming breaks down complex tasks into smaller subtasks. It then models problems as workflows of sequential decisions made in discrete time steps. Each decision is made based on the possible resulting next state, since dynamic programming considers potential future states and reward signals when selecting actions. Indeed, dynamic programming is a value-based method, meaning it selects actions based on their estimated values according to a policy that aims to maximize its value function.

An agent's reward r for a given action is defined as a function of that action a , the current environmental state s , and the potential next state s' :

$$r_t(s, a) = \sum_{s' \in S} r_t(s, a, s') p_t(s' | s, a)$$

where $P_t(s' | s, a)$, called the *transition probability function*, denotes the probability that the system is in state s' at time $t+1$ when the agent chooses action a in state s at time t [12].

Dynamic programming seeks to maximize cumulative rewards by consistently selecting the best possible actions at each state, using recursive relationships such as the Bellman equations [11]. Dynamic programming theory guarantees the existence of at least one optimal stationary policy. However, classical dynamic programming techniques require full knowledge of the reward function and transition probabilities, which are generally unavailable in real-world scenarios.

Model-Free Reinforcement Learning

Monte Carlo methods The Monte Carlo method assumes a black-box environment, making it a model-free approach. Since no prior knowledge of the environment is available, Monte Carlo methods rely entirely on direct interaction to gather information, so these methods are purely experience-driven. These methods sample sequences of states, actions and rewards exclusively through interaction with the environment. As a result, learning occurs through trial and error [12] rather than through predefined probability distributions or transition models. This experiential learning process enables Monte Carlo methods to adapt to unknown environments, but it also requires a large number of episodes to achieve reliable performance. Monte Carlo methods estimate value functions by computing the average return for each state-action pair based on observed episodes. Consequently, Monte Carlo methods must wait until an entire episode (or planning horizon) is completed before updating their value estimates and policies. Within this framework, two main variants can be distinguished: First-Visit and Every-Visit Monte Carlo.

- **First-Visit Monte Carlo** estimates the value of a state by averaging the returns obtained only on the first visit to that state in each episode. This avoids updating the same state multiple times within the same episode, reducing the risk of bias in the results.
- **Every-Visit Monte Carlo**, on the other hand, considers all visits to a state within each episode. The state’s value is estimated by averaging the returns obtained in each visit. This method uses more data than First-Visit but may introduce greater correlation between observations.

In both variants, the value function is updated according to:

$$V(S_t) \leftarrow V(S_t) + \alpha [G_t - V(S_t)]$$

where $V(S_t)$ is the estimated value of state S_t , α is the learning rate, and G_t is the actual return obtained from time step t .

Temporal Difference learning

2.4.3 Deep Q-Networks

2.5 Atari-like Environments as Testbeds

2.5.1 Characteristics and Challenges of 2D Game Environments

3 State of the Art and Challenges

3.1 Challenges in Symbolic Interpretation

3.1.1 The Illusion of Understanding

The Lack of World Models

3.1.2 The Black Box Problem

Making Sense of AI: Toward Explainability

3.1.3 Learning Paradigms and Explainable AI

Learning Without Understanding

Explainable AI: Goals and Approaches

Post-hoc Explainability

Intrinsic Interpretability

Why XAI is Not Enough

3.2 Cognitive Foundations

3.2.1 Human World Modelling and the Origins of Structured Cognition

3.2.2 Core Knowledge Theory: Cognitive Principles and Psychological Foundations

3.2.3 Definition and Relevance for World Understanding

3.2.4 Cognitive Models and Symbolic Representation

3.3 Positioning of the Proposed Framework

4 Framework Design - Proposed Framework

4.1 Conceptual Foundations

4.1.1 Evolution of the Idea

4.2 Proposed Implementation

4.2.1 Segmentation

4.2.2 Symbolic System as Knowledge Construction Module

4.2.3 From Symbolic Rules to a Playable Environment

4.2.4 Generic Environment as a Transitional Representation

4.2.5 Deep Q-Network in a Domain-Agnostic Environment

5 Implementation

5.1 End-to-End Pipeline

5.2 General Architecture of the Framework

5.2.1 Design Choices

5.2.2 Selected Techniques and Algorithms

5.2.3 Modular Structure of the System

Video Acquisition and Preprocessing Module

Feature Extraction Module

Symbolic Mapping Module - Symbolic Reasoning Engine

Integration of Core Knowledge for Semantic Understanding

RL Agent

5.2.4 Optimizations and Efficiency Considerations

6 Experimental Evaluation

6.1 Datasets and Testing Scenarios

6.1.1 Evaluation Metrics: Accuracy, Interpretability, Scalability

6.2 Experimental Results

6.2.1 Comparison with Existing Methods

6.2.2 Critical Discussion of the Results

7 Applications and Future Perspectives – Conclusion

7.1 Future Directions

7.1.1 Potential Applications of the Framework

7.1.2 Possible Extensions of the Work

7.1.3 Open Challenges and Future Research Directions

7.2 Summary of Contributions, Impact of the Research, and Final Reflections

References

Bibliography

- [1] B. Liang, Y. Wang, and C. Tong, “Ai reasoning in deep learning era: From symbolic ai to neural-symbolic ai,” *Mathematics*, vol. 13, no. 11, p. 1707, 2025.
- [2] G. Marcus, “Deep learning: A critical appraisal,” *arXiv preprint arXiv:1801.00631*, 2018. [Online]. Available: <https://arxiv.org/abs/1801.00631>
- [3] J. Schmidhuber, “Deep learning in neural networks: An overview,” *Neural Networks*, vol. 61, pp. 85–117, 2015.
- [4] A. S. d’Avila Garcez, L. C. Lamb, and D. M. Gabbay, *Neural-Symbolic Cognitive Reasoning*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009. [Online]. Available: <https://link.springer.com/book/10.1007/978-3-540-73246-4>
- [5] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” *Nature*, vol. 521, pp. 436–444, 2015. [Online]. Available: <https://doi.org/10.1038/nature14539>
- [6] A. S. d’Avila Garcez, T. R. Besold, L. De Raedt, P. Földiák, P. Hitzler, T. Icard, and D. L. Silver, “Neural-symbolic learning and reasoning: Contributions and challenges,” in *AAAI Spring Symposia*, Mar. 2015, pp. 18–21. [Online]. Available: <https://www.aaai.org/Library/Symposia/Spring/ss15-01.php>
- [7] B. M. Lake, T. D. Ullman, J. B. Tenenbaum, and S. J. Gershman, “Building machines that learn and think like people,” *Behavioral and Brain Sciences*, vol. 40, p. e253, 2017.
- [8] L. P. Kaelbling, M. L. Littman, and A. W. Moore, “Reinforcement learning: A survey,” *Journal of Artificial Intelligence Research*, vol. 4, pp. 237–285, 1996.

- [9] M. L. Puterman, *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. Hoboken, NJ, USA: Wiley, 2014.
- [10] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*. Englewood Cliffs: Prentice-Hall, 1995.
- [11] V. Mnih *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529–533, 2015.
- [12] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. MIT Press, 2015.

A Additional Technical Details

B Code Snippets or Pseudocode of Relevant Modules

C Dataset Specifications or Experimental Settings